

字节前端面经知识点分类汇总

来源：字节前端面经 1~7.md，原文内容未做任何改动，仅按知识体系重新分类整理。

一、JavaScript 基础

1.1 类型与值

- `null` 和 `undefined` 的区别：含义、类型、场景
- `null == undefined` 为什么是 `true`
- 六个输出：`null` 转数字是 0，`undefined` 转数字是 NaN
- 暂时性死区（TDZ）：`let/const` 在声明前不能使用
- 暂时性死区产生原因
- 哪些声明方式有暂时性死区

1.2 函数与执行机制

- 微任务和宏任务：定义、代表、执行时机
- 微任务宏任务输出顺序：同步 → `await` 后微任务 → `.then` 微任务 → `setTimeout` 宏任务
- 微任务宏任务解决什么问题：避免单任务卡死主线程，插入渲染机会
- `Promise` 三种状态：`pending`、`fulfilled`、`rejected`，状态不可逆
- 生成器（`Generator`）和迭代器（`Iterator`）：`next()`、`yield`、惰性求值、异步流程控制

1.3 异步编程

- 流式输出/SSE：`EventSource`、单向推送、自动重连、`retry` 控制
- 流式数据解码乱码解决：`TextDecoder` + `stream` + 缓冲区
- 流式 Markdown 容错：自动补全未闭合代码块
- 流式 Markdown 渲染：流式场景下不全文重解析
- SSE vs `WebSocket` 本质区别：单向 vs 双向、HTTP vs 独立协议
- DeepSeek 用 SSE vs 聊天用 `WebSocket` 的原因
- `WebSocket` 长时间不发言浪费吗：空闲断开优化
- 指数退避重连：`1s → 2s → 4s → 8s`，防止雪崩
- `fetchWithRetry`：`fetch` + 重试逻辑 + 指数退避

1.4 原型与继承

- 原型链相关（见 `null` 的场景）

1.5 语法与 API

- for...in vs for...of: 遍历键名 vs 遍历值、对象 vs 可迭代对象
- 普通对象能被 for...of 遍历吗: 需实现 [Symbol.iterator]
- 闭包: 函数访问外部变量, 外部函数执行完仍可访问
- 事件冒泡和捕获: 传播方向、addEventListener 第三个参数、执行顺序
- 事件委托: 统一绑父元素, 通过 event.target 判断

1.6 迭代器与生成器

- Iterator: 对象有 next() 返回 {value, done}, Array/Map/Set 内置
- Generator: function* + yield, 创建迭代器的快捷方式

二、浏览器与网络

2.1 网络基础

- TCP vs UDP: 连接、可靠、顺序、速度; TCP 适用文件/网页, UDP 适用直播/DNS
- 浏览器输入 URL 到展示页面过程: DNS → TCP握手 → HTTP请求 → 响应 → DOM树 → CSSOM → 渲染树 → 布局 → 绘制 → 合成
- script 标签阻塞: async/defer/放 body 底部
- CSS 阻塞: CSSOM 未完成阻塞渲染和 JS 执行; 关键 CSS 内联 + 非关键异步
- async 和 defer 区别: async 下载完立即执行顺序不确定, defer 等 DOM 解析完按序执行
- 502 Bad Gateway: 上游服务器返回无效响应
- 504 Gateway Timeout: 网关请求上游超时

2.2 跨域

- 同源策略: 协议/域名/端口任一不同即跨域
- 跨域解决方案: CORS、代理、Nginx 反向代理、JSONP、postMessage、WebSocket
- CORS: 跨域资源共享, 后端设 Access-Control-Allow-Origin 响应头
- 为什么设置跨域限制: 防止恶意网站 AJAX 偷取 Cookie/Token

2.3 缓存

- 强缓存: Cache-Control: max-age / Expires, 缓存期内不发请求
- 协商缓存: Last-Modified / ETag, 缓存过期询问服务器
- 前端存储: Cookie (4KB 每次请求带)、LocalStorage (5MB 持久)、SessionStorage (5MB 会话)、IndexedDB (大容量异步)
- 静态资源缓存: contenthash 长效缓存; 图片雪碧图/SVG sprite/CDN; 字体分包; HTML 不缓存

2.4 浏览器渲染与性能

- 重排 (Reflow) : 重新计算布局 (位置、尺寸) , 成本高
- 重绘 (Repaint) : 重新上色 (颜色、可见性) , 成本低
- transform 为什么不触发重排/重绘: 合成层走 GPU, 跳过主线程
- position 改变触发重排: 改变布局规则
- 伪元素触发重排还是重绘: 改颜色只重绘, 改宽高触发重排
- 浏览器帧数: 60fps 每帧约 16.6ms, 超过则掉帧
- rAF (requestAnimationFrame) : 下一帧前执行, 跟随屏幕刷新率
- rIC (requestIdleCallback) : 浏览器空闲时执行, 低优先级任务
- Web Worker: 后台线程执行 JS, 不阻塞主线程, 不能操作 DOM, postMessage 通信
- React Streaming SSR: 边渲染边输出 HTML, 配合 Suspense

2.5 浏览器资源加载

- prefetch vs preload: preload 立即高优先级, prefetch 空闲时低优先级
- 图片懒加载: IntersectionObserver 监听 / 原生 loading="lazy" / scroll + getBoundingClientRect
- 多行溢出: -webkit-line-clamp / 伪元素覆盖
- 单行溢出: white-space + overflow + text-overflow
- 浏览器兼容性降级: IntersectionObserver 降级 scroll + getBoundingClientRect

三、框架 (Vue & React)

3.1 Vue 核心

- Vue2 vs Vue3: Object.defineProperty vs Proxy、Options API vs Composition API、性能、TS、Fragment
- Vue3 Proxy 原理: 拦截读/写/删除, 惰性代理, 数组索引、新增/删除属性都响应
- Vue2 痛点 vs Proxy 解决: 数组下标、新增属性、delete、需递归劫持
- Vue Diff 算法: 同层比较、双端对比 (头头/尾尾/头尾/尾头) 、key 优化、Vue3 最长递增子序列
- nextTick: Vue 异步 DOM 更新后执行, 微任务队列

3.2 Vue 生态

- Pinia 使用和原理: 多个独立 reactive(), 闭包单例, useXxxStore() 获取
- Vuex vs Pinia: 嵌套模块 vs 扁平化、需要 mutations vs 直接改 state、弱 TS vs 完美支持

3.3 React 核心

- React 认识: 声明式 UI、JSX、虚拟 DOM、单向数据流、Fiber 可中断渲染、并发模式
- React Fiber: 16 架构重写协调引擎, 任务拆小单元可暂停/恢复/丢弃, 优先级调度
- 受控组件 vs 非受控组件: 数据由 state 管理 vs DOM 自己管理, 通过 ref 取值
- 受控/非受控场景: 受控用于实时校验、共享数据、禁用修改、autocomplete; 非受控用于简单表单、高频输入

入（游戏/编辑器）

- useEffect 依赖数组/对象的比较：浅比较（Object.is），每次创建新引用会重复执行，用 useMemo 缓存
- hooks 规则：只能在函数组件/自定义 hook 顶层调用，不能在条件/循环中
- React 流式传输：renderToPipeableStream，边渲染边输出，Suspense 异步组件

3.4 框架性能

- 虚拟 DOM：JS 对象模拟真实 DOM，先 diff 再最小化更新
- 虚拟 DOM 优势：跨平台（Web/Native/小程序）、开发体验、函数式编程
- 虚拟 DOM 不适用：大量静态内容、对性能极致要求的场景
- Diff 算法：同层比较，复杂度 $O(n)$ 而非 $O(n^3)$

3.5 跨端框架

- Taro 原理：编译原理，React/Vue → AST → 各平台代码（小程序/H5/RN）
- Expo 原生交互：封装好的 API，底层走 RN Bridge

四、工程化（Webpack & 构建）

- Webpack 功能：代码分割、Tree Shaking、HMR、热更新、压缩、哈希文件名缓存、Loader、Plugin、别名、环境变量、CDN 依赖排除
- 代码分割：import() 动态导入，路由懒加载
- Tree Shaking：删除未使用的 export
- HMR：改代码不刷新页面
- 减小打包体积：代码分割、Tree Shaking、压缩、CDN externals、图片压缩、WebP、splitChunks 提取公共依赖
- CI/CD 构建流程：git push → CI → npm install → npm run build → 产物上传 OSS/CDN → 部署 → 健康检查
- 按需加载：代码分割 + 懒加载，路由级别 / 组件级别 / 条件加载

五、CSS

5.1 盒模型与布局

- transform: translateY() 不影响其他元素：transform 不改变文档流，只视觉偏移
- 三个 div 默认表现：块级元素独占一行，宽 100% 父容器，高由内容撑开
- BFC 解决的问题：外边距折叠、清除浮动、防止被浮动覆盖
- 触发 BFC 的方式：overflow hidden/auto/scroll、display flow-root/flex/grid/inline-block/table-cell、position absolute/fixed、float、contain、html 根元素

5.2 视觉效果

- CSS 伪元素：清除浮动、装饰图标、placeholder 样式、列表圆点、tooltip 小三角
- CSS 自定义变量 (--xxx)：运行时可动态修改，主题切换、响应式、组件变体

5.3 性能

- transform/opacity 的渲染优化：合成层走 GPU，不触发重排重绘

六、项目实践 (Chat 项目)

6.1 流式输出

- SSE/流式输出：EventSource 使用、重连机制、retry 控制、主动 close()
- 流式数据解码：TextDecoder + stream + 缓冲区处理多字节截断
- Markdown 流式渲染：正则/marked/markdown-it 解析，自动补全未闭合语法
- 多语言/UTF-8：变长编码 (ASCII 1B/中文 3B/emoji 4B)，TextDecoder stream 处理；RTL 靠 CSS direction + 逻辑属性
- 缓冲区大小限制：最大 10KB 上限，防止异常数据撑爆内存
- Markdown 渲染插件：代码高亮、数学公式 KaTeX、流式友好

6.2 项目问题排查

- 流式乱码问题解决：TextDecoder + stream + 自己实现缓冲区
- Markdown 渲染闪烁/报错：流式语法不完整导致；自动补全未闭合代码块
- 长对话历史性能：状态膨胀、滚动性能下降
- 性能问题发现：列表超 200 条卡顿掉帧，Chrome DevTools Performance 面板录制
- 性能问题排查：Elements 看 DOM 节点数 → Performance 找长任务 → React DevTools Profiler → memory 面板内存快照

6.3 登录鉴权

- JWT 双 token：access token (短期 15 分钟) + refresh token (长期 7 天)
- 为什么双 token：access 被盗影响有限；refresh 可存 httpOnly cookie 防 XSS，可服务端吊销

6.4 编辑器

- 受控 vs 非受控在编辑器的选择：编辑器状态极复杂，倾向于非受控，contentEditable + DOM 操作，只在提交时同步值 (如 Slate.js、Quill)
 - 理想 NPM 编辑器组件 API：defaultValue 非受控为主 + 可选 value 受控模式
-

七、性能优化

7.1 列表与渲染

- 长列表优化：后端分页、无限滚动 (IntersectionObserver)、虚拟列表 (只渲染可视区)、图片懒加载
- 虚拟列表原理：计算 startIndex/endTimeIndex, slice 只渲染可见区, padding 占位撑滚动条；固定高度简单计算，非等高需 positions 数组 + 二分查找
- 虚拟列表划入划出图片反复请求：HTTP 缓存、前端 Map 缓存已加载 URL、Service Worker 拦截缓存
- 虚拟列表性能问题怎么发现：DevTools Performance 面板看 Layout/Paint，帧率低于 30fps
- Diff 算法中 key 为什么不能用 index：列表顺序变化时复用错误节点，DOM 更新错乱或输入框内容错位

7.2 网络与通信

- 短连接方案调研：短轮询 (延迟高浪费带宽)、长轮询 (服务器压力大)、WebSocket (延迟低省带宽)
- WebSocket 空闲断开优化：5 分钟空闲断开，发消息重连
- SSE 自带重连：错误时自动重连，服务端可发 retry 控制间隔

7.3 请求优化

- input 框发多次请求，第一次比第二次慢：用请求序列号或 AbortController，abort 上一个未完成的请求
- 防抖节流应用：防抖 (搜索框停手 300ms)、节流 (滚动/mousemove/resize)
- 防抖节流原理：闭包保存 timer

7.4 图片优化

- 图片压缩：image-webpack-loader / imagemin 压缩；大图转 WebP；小图转 base64 内联减少请求；用户上传图片后端用 sharp/GraphicsMagick
- 图片懒加载：IntersectionObserver / 原生 loading="lazy" / scroll + getBoundingClientRect

八、Native 与跨端

8.1 Web 与原生交互

- JS Bridge：URL Scheme 拦截、注入 window.bridge 对象、消息队列
- Expo 原生交互：封装好的 API，底层走 RN Bridge；Expo vs 裸 RN 区别

8.2 跨端框架

- Taro 原理：编译原理 AST 转换，多端输出

九、运维与部署

- CI/CD 流程：git push → CI/CD (GitHub Actions/Jenkins) → npm install → npm run build → 产物上传 OSS/CDN → 部署 Nginx/Docker → 健康检查
- Nginx 反向代理：同域转发到不同服务，解决跨域 (生产环境)

十、其他

10.1 Git

- git merge vs git rebase: merge 保留完整历史有分叉, rebase 历史直线但改变提交历史

10.2 业务场景

- 刷新页面倒计时不重置: 后端记录 startTime, 前端计算 考试时长 - (now - startTime)
- 缓存清了怎么办: 真正的数据同步后端, 清缓存后从接口重新拉
- 做 Chat 项目动机: 深入理解流式输出、大模型 API、Markdown 渲染、状态管理
- 有挑战的卡点: 流式乱码、Markdown 不完整闪烁、长对话状态膨胀滚动下降

10.3 Canvas

- Canvas: JS 画图的 HTML 标签, 像素级画布, 可画图形/动画/处理图片/图表/游戏

共涵盖 **10 大分类**, 来源原文内容完整保留, 可直接用于面试复习。